



MICROSERVICIOS

Prof. José Rendón

DESCRIPCIÓN BREVE

Tarea de investigación sobre los microservicios, concepto, tipos de arquitectura, ejemplos de aplicación y casos de éxito.

Jonathan Paladines Navarrete

Tendencias Actuales de Programación – 5 “A”

Contenido

MICROSERVICIOS	2
¿Qué son los microservicios?	2
Arquitectura de microservicios.....	3
Ejemplos de Microservicios	5
Casos de éxito con microservicios	6
Características del Software de microservicios.	8
Referencias	10

MICROSERVICIOS

¿Qué son los microservicios?

Los microservicios son tanto un estilo de arquitectura como un modo de programar software.

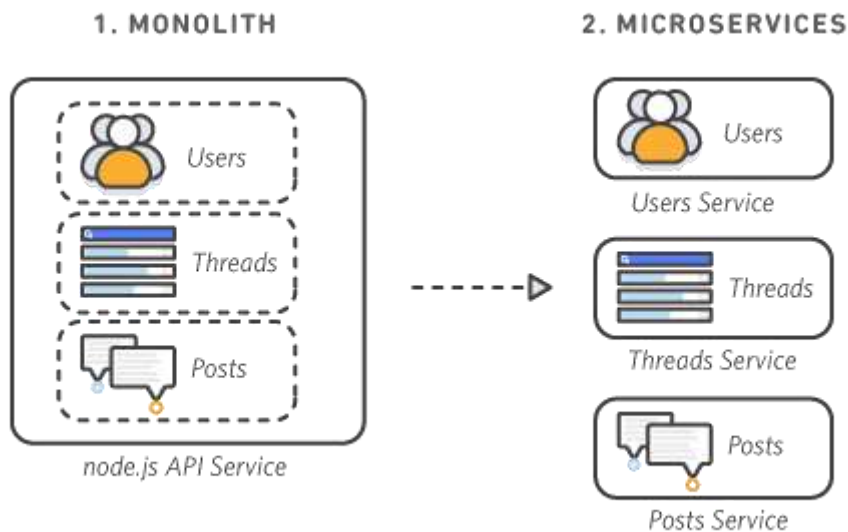
Con los microservicios, las aplicaciones se dividen en sus elementos más pequeños e independientes entre sí. A diferencia del enfoque tradicional y monolítico de las aplicaciones, en el que todo se compila en una sola pieza, los microservicios son elementos independientes que funcionan en conjunto para llevar a cabo las mismas tareas. Cada uno de esos elementos o procesos es un microservicio.

El software está compuesto por pequeños servicios independientes que se comunican a través de API bien definidas. Los propietarios de estos servicios son equipos pequeños independientes.

Las arquitecturas de microservicios hacen que las aplicaciones sean más fáciles de escalar y más rápidas de desarrollar. Esto permite la innovación y acelera el tiempo de comercialización de las nuevas características.

Arquitectura de microservicios

Con una arquitectura de microservicios, una aplicación se crea con componentes independientes que ejecutan cada proceso de la aplicación como un servicio. Estos servicios se comunican a través de una interfaz bien definida mediante API ligeras. Los servicios se crean para las capacidades empresariales y cada servicio desempeña una sola función. Debido a que se ejecutan de forma independiente, cada servicio se puede actualizar, implementar y escalar para satisfacer la demanda de funciones específicas de una aplicación.



Por ello para aplicar la nueva arquitectura que proponen los microservicios, sobre todo en soluciones IT ya desarrollada, deberemos actuar con una arquitectura en dos capas. Una capa que actúe de manera interna y la segunda capa de manera externa.

Algunas de las herramientas más usadas para la creación de una arquitectura de microservicios son:

- Amazon Simple Queue Service (SQS)
- Apache Kafka
- Apache Kafka
- Kubernetes
- Elixir

- Spring Boot
- Google Cloud Functions
- AWS Lambda
- Microsoft Azure Functions

La idea principal detrás de la arquitectura de microservicios es que algunos tipos de aplicaciones se vuelven más fáciles de construir y mantener cuando se dividen en piezas más pequeñas que se pueden componer y que funcionan juntas. En otras palabras, cada componente se desarrolla por separado y la aplicación es simplemente la suma de sus componentes constituyentes. En una arquitectura de microservicios, cada servicio ejecuta un proceso único y, por lo general, administra su propia base de datos. Esto no solo brinda a los equipos de desarrollo un enfoque más descentralizado para crear software, sino que también permite que cada servicio se implemente, reconstruya, vuelva a implementar y administre de forma independiente.

En conclusión, la filosofía de la arquitectura de microservicios: «Haz una cosa y hazla bien». Los servicios pueden ejecutarse dentro del mismo proceso, pero deben poder implementarse de forma independiente y ser fáciles de reemplazar. Se pueden implementar utilizando diferentes lenguajes de programación, bases de datos y entornos de software. Los servicios son pequeños, detallados para realizar una sola función. Abarcan la automatización de las pruebas y la implementación, el proceso de desarrollo de software de entrega continua, fallas y fallas, similares a los sistemas anti-frágiles. Cada servicio es elástico, resistente, componible, mínimo y completo.

Ejemplos de Microservicios

Se realizará una Tienda de Libros o Cursos donde se mostraran los artículos disponibles, el cliente podrá obtener información de cualquier artículo ingresando su email. Un cliente podrá ver los artículos que obtuvo solo proporcionando su correo.

Diagrama

Para este pequeño ejemplo utilizaremos cuatro microservicios :

- **Api Gateway.**- Tiene la tarea de comunicarse con el frontend y según las peticiones de el usuario se comunica con los otros microservicios.
- **Users y Courses.**- Dos microservicios que están en un nivel atómico estos solo se encargan de ellos. Notemos que cada microservicio tienen su propia base de datos.
- **Email.**- Microservicio encargado solo de enviar correos a los usuarios, notemos que no tiene una comunicación REST API como en los anteriores microservicios, para este utilizaremos el protocolo AMQP ayudándonos con RabbitMQ, esto nos dará la capacidad de poner en cola las peticiones que le estén llegando.

Flujo

Se mostrará una variedad de libros o cursos de los cuales el usuario podrá seleccionar alguno, si desea obtener información de algún artículo entonces procedemos a pedirle su Correo para mandarle la información del artículo que selecciono ,si el Usuario es nuevo entonces se le enviara un correo de bienvenida a la aplicación, posteriormente se le enviaremos la información del artículo que selecciono.

Diagrama de Secuencia para Solicitar información de un artículo

Descripción

- **BuyCourse:** Desde el cliente se hace la solicitud de información de un artículo a el servicio Gateway.
- **GetDataUser:** El servicios Gateway solicita los datos del Usuario al servicio Users enviando el email del cliente ,este servicio verifica si el usuario existe ,si no existe el

usuario crea uno nuevo con el email proporcionado , teniendo el usuario respondemos a el servicio Gateway enviando el usuario.

- **SendWelcomeUser:** El servicio Gateway verifica si el usuario obtenido a sido creado recientemente si es así se envía un correo de Bienvenida a la Tienda.
- **SaveBuyCourse:** El servicio Gateway se comunica con el servicio Courses ,este guarda los datos de la solicitud de información de el articulo por usuario, al finalizar el guardado del curso se manda una respuesta de exitosa a Gateway.
- **SendEmailByBuy:** Una vez realizada la solicitud se realiza el envío de un email al cliente con la información del artículo.

Este solo es un flujo de una funcionalidad que se puede realizar con la aplicación. Seguramente pensaron pero si esta aplicación se podría hacer de una forma mas sencilla con una arquitectura monolítica y yo les diría “Pues si ustedes tienen razón” pero recordemos que este es un ejemplo, generalmente la arquitectura de microservicios se aplica para aplicaciones de gran crecimiento esto nos ayudara a modularizar cada funcionalidad que necesitemos, y cada módulo será independiente a cualquier otro modulo.

Casos de éxito con microservicios

No hay mejor forma de conocer el alcance que ha tenido este método de desarrollo que ver quiénes lo han implementado. Multitud de webs que sirven aplicaciones a gran escala han decidido invertir en la evolución hacia los microservicios en vistas de un futuro donde el mantenimiento y escalabilidad de sus productos es mucho más simple, efectivo y rápido. Vamos a destacar algunas de estas compañías, que lo mismo hasta os suenan:

Netflix : Esta plataforma tiene una arquitectura generalizada que desde hace ya un par de años (coincidiendo con su “boom” en U.S.A.) se pasó a los microservicios para el funcionamiento de sus productos. A diario recibe una media de mil millones de llamadas a sus diferentes servicios (se dice que es responsable del 30% del tráfico de Internet) y es capaz de adaptarse a más de 800 tipos de dispositivos mediante su API de streaming de vídeo, la cual para ofrecer un servicio más estable, por cada solicitud que le pedimos, ésta realiza cinco solicitudes a diferentes servidores para no perder nunca la continuidad de la transmisión.

Amazon : No soporta tantos dispositivos como Netflix, pero tampoco es que sea fundamental para cubrir su sector. Migró hace tres años a la arquitectura de microservicios siendo una de las primeras grandes compañías que la implementaban en producción. No hay cifra

aproximada de la cantidad de solicitudes que pueden recibir a diario, pero no son pocas. Entre

éstas encontramos multitud de aplicaciones, las API del servicio web que ofrecen o la propia web de Amazon, cuyos ingenieros reconocen que habría sido imposible sobre la arquitectura monolítica con la que trabajaban previamente.

Ebay : Cómo no, una de las empresas con mayor visión de futuro, siendo pionera en la adopción de tecnologías como Docker o ésta que nos ocupa. Su aplicación principal comprende varios servicios autónomos, y cada uno ejecutará la lógica propia de cada área funcional que se ofrece a los clientes.

Walmart revitalizó con éxito su arquitectura fallida con microservicios

Este es un buen ejemplo de lo que se debe hacer cuando la arquitectura obsoleta comienza a afectar negativamente al negocio. Esta es la pregunta multimillonaria que el Departamento de TI de Walmart Canadá tuvo que abordar después de que fallaran en los Black Friday durante dos años seguidos.

La migración a microservicios en realidad trajo resultados notables :

- Las conversiones aumentaron un 20 % literalmente de la noche a la mañana
- Los pedidos móviles aumentaron un 98 % al instante
- Sin tiempo de inactividad en Black Friday o Boxing Day
- Los ahorros operativos también fueron significativos, ya que la empresa pasó de su costoso hardware a hardware básico.
- Ahorraron el 40 % de la potencia de cómputo y experimentaron un ahorro de costos total del 20 al 50 %.

Spotify crea una experiencia de usuario excepcional con microservicios

Kevin Goldsmith, vicepresidente de ingeniería de Spotify, sabe por experiencia que una empresa que pretende moverse rápido y mantenerse innovadora en un mercado altamente competitivo requiere una arquitectura que pueda escalar.

Spotify atiende a más de 75 millones de usuarios activos por mes, con una duración promedio de sesión de 23 minutos, mientras ejecuta roles comerciales increíblemente complejos entre bastidores.

Entonces, Spotify llegó a la conclusión de que si le preocupa escalar a cientos de millones de usuarios, debe construir su sistema de manera que escale los componentes de forma independiente. Y Spotify construyó una arquitectura de microservicios con equipos full-stack autónomos a cargo para evitar el infierno de sincronización dentro de la organización. Estos equipos son autónomos y su misión no se superpone con la misión de otros equipos.

Ahora, Spotify tiene alrededor de 90 equipos, 600 desarrolladores y 5 oficinas de desarrollo en 2 continentes que crean el mismo producto, por lo que necesitaban minimizar estas dependencias tanto como fuera posible.

Lo que realmente le gusta a Spotify de los microservicios es que no tienen grandes fallas; Los grandes servicios fallan a lo grande, los servicios pequeños fallan a lo pequeño. La construcción de una arquitectura de microservicios le permite a Spotify tener una gran cantidad de servicios inactivos al mismo tiempo sin que los usuarios lo noten. Construyeron su sistema asumiendo que los servicios pueden fallar todo el tiempo, por lo que los servicios individuales que podrían estar fallando no están haciendo demasiado, por lo que no pueden arruinar la experiencia de usar Spotify.

Características del Software de microservicios.

Como decía no hay un estándar en estas arquitecturas, pero sí que podemos destacar varias características comunes en mayor o menor medida:

1. El software construido en base a microservicios se podrá descomponer en varias partes funcionales independientes. Siendo así, cada uno de estos servicios podrá ser desplegado, modificado y re-desplegado sin comprometer los otros aspectos funcionales de la aplicación; y como resultado en caso de necesitarlo, sólo tendremos que modificar un par de servicios en lugar de redespargar toda la aplicación al completo nuevamente.
2. La forma en la que se organizan los microservicios suele ser en torno a las necesidades, capacidades y prioridades del cliente o negocio en el que se implantará. A diferencia de un entorno monolítico donde cada equipo de trabajo tiene un enfoque específico sobre un apartado de la aplicación, en la arquitectura de microservicios se utilizan

módulos multifuncionales, adaptando así un módulo común a todos para que ofrezca un servicio determinado. El ahorro en tiempo de desarrollo es inmenso, por no hablar de la comodidad a la hora de programar tareas de mantenimiento, donde podemos revisar un módulo mientras el resto del equipo de trabajo no ve interrumpida su jornada.

3. El funcionamiento del software de microservicios puede parecerse al sistema de trabajo clásico de UNIX (se recibe una petición, se procesa y se genera una respuesta consecuente. Al contrario que los entornos ESB (Enterprise Service Buses, o Bus de servicio empresarial) donde se utilizan equipos para enrutar mensajes, redireccionar tráfico, aplicar reglas de denegación de acceso etc... Se podría decir que la arquitectura de microservicios cuenta con puntos finales “inteligentes” que procesan la información en término y aplican la lógica establecida por el desarrollador.
4. La arquitectura de microservicios mantiene un sistema similar a un gobierno descentralizado , donde cada módulo contará por ejemplo con su propia base de datos, en lugar de acudir todos a la misma sobrecargándola así de solicitudes y arriesgándonos a que si falla ésta, todas la aplicación caiga.
5. Cuando varios servicios están comunicados entre sí, por lo general contarán con un sistema de aviso y actuación si alguno de éstos servicios llega a fallar (como mostrar una advertencia, enviar un mail a soporte, avisar a los usuarios de un fallo temporal, etc...), filtrando adecuadamente la información destinada a este módulo y favoreciendo la correcta gestión de los recursos entre los módulos funcionales restantes.

Referencias

- Alvarez Caules, C. (23 de 01 de 2015). *¿Qué es un Microservicio?* Obtenido de <https://www.arquitecturajava.com/que-es-un-microservicio/>
- Garzas, D. J. (19 de 06 de 2015). *¿Qué es eso de los microservicios?* Obtenido de <https://www.javiergarzas.com/2015/06/microservicios.html>
- Mamani Martinez, Z., Del Pino Rodriguez, L., & Gonzalez Suarez, J. (2020). Arquitectura basada en Microservicios y DevOps para una ingeniería de software continua. *Industrial Data*, 141-149.
- Murcia, R. K. (20 de 01 de 2022). *Keep Coding - Tech School*. Obtenido de <https://keepcoding.io/blog/que-son-los-microservicios/>